МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Кафедра «Информационная безопасность систем и технологий»

Отчет

о лабораторной работе №4

на тему «Исследование планирования процессов в ос linux»

Дисциплина: Операционные системы

Группа: 22ПИ1

Выполнил: Д.А. Шапырин

Количество баллов:

Дата сдачи:

Принял: П.П, Петров

2026

1 Цель работы: изучить механизмы планирования процессов в операционной системе Linux, исследовать влияние значения nice на приоритет и фактическое распределение процессорного времени, ознакомиться с политиками планирования реального времени.

2 Задание к лабораторной работе:

2.1 Изучить теоретические сведения о планировании процессов в ОС Linux: классы и политики планирования, статические и динамические приоритеты, механизм nice.

2.2 Исследовать приоритеты процессов в работающей системе с помощью утилиты ps.

2.3 Продемонстрировать изменение приоритета процесса утилитами nice и renice.

2.4 Определить диапазоны приоритетов, поддерживаемые политиками плани-рования, с помощью утилиты chrt.

2.5 Экспериментально оценить влияние значения nice на распределение про-цессорного времени между конкурирующими процессами.

2.6 Оформить отчёт по лабораторной работе.

3 Результаты работы:

3.1 Были изучены теоретические сведения о планировании процессов в ОС Linux: классы и политики планирования, статические и динамические приоритеты, механизм nice.

3.1.1 Планирование процессов

Планирование процессов – одна из ключевых функций операционной системы: планировщик решает, какой из готовых к выполнению процессов получит процессор и на какое время. От алгоритма планирования зависят отзывчивость интерактивных задач, пропускная способность системы и справедливость распределения ресурсов между задачами .

В многозадачной системе с вытесняющим планированием ядро может в любой момент приостановить выполняющийся процесс и передать процессор

другому. Решение принимается на основе класса планирования процесса, его приоритета и накопленной статистики использования процессора .

Смена выполняющегося процесса требует переключения контекста: яд-ро сохраняет состояние регистров и адресного пространства текущего процес-са и восстанавливает состояние следующего. Переключение контекста не яв-ляется бесплатным – помимо прямых затрат на сохранение и восстановление состояния, оно приводит к вытеснению данных из кэшей процессора и буфера TLB. Поэтому планировщик всегда ищет компромисс между отзывчивостью (частые переключения, малые кванты) и накладными расходами (редкие переключения, длинные кванты) .

С точки зрения планировщика процессы принято делить на два класса поведения: ограниченные процессором (CPU-bound), которые используют весь предоставленный квант для вычислений, и ограниченные вводом-выводом (I/O-bound), которые бóльшую часть времени ожидают завершения операций и лишь эпизодически нуждаются в процессоре. Хороший плани-ровщик универсальной ОС должен обеспечивать низкую задержку пробужде-ния интерактивным (I/O-bound) задачам, не допуская при этом голодания вы-числительных задач .

3.1.2 Классы и политики планирования в Linux

Ядро Linux реализует несколько классов планирования, каждый из которых объединяет одну или несколько политик:

- класс справедливого планирования – политики SCHED_OTHER (обычные процессы), SCHED_BATCH (пакетные задачи) и SCHED_IDLE (задачи с минимальным приоритетом);
- класс реального времени – политики SCHED_FIFO (выполнение до добровольной уступки или вытеснения более приоритетным процессом) и SCHED_RR (циклическое выполнение с квантом времени);
- класс SCHED_DEADLINE – планирование по алгоритму EDF (Earliest Deadline First) с резервированием процессорного времени.

Процессы реального времени имеют статический приоритет от 1 до 99 и всегда вытесняют обычные процессы. Обычные процессы имеют статический приоритет 0, а конкуренция между ними разрешается справедливым планировщиком: исторически это CFS (Completely Fair Scheduler), в современных ядрах – его развитие EEVDF.

Справедливый планировщик не использует фиксированных квантов времени. Каждому процессу сопоставляется виртуальное время выполнения (vruntime) – фактически полученное процессорное время, нормированное на вес процесса: у «тяжёлых» (высокоприоритетных) процессов vruntime растёт медленнее, у «лёгких» – быстрее. На выполнение выбирается процесс с наименьшим виртуальным временем, благодаря чему за достаточно длинный интервал каждый процесс получает долю процессора, пропорциональную своему весу . Алгоритм EEVDF (Earliest Eligible Virtual Deadline First), заменивший CFS начиная с ядра 6.6, дополняет эту схему понятиями допустимо-сти (eligibility) и виртуального срока (virtual deadline), что позволяет строже ограничивать задержку планирования для задач, чувствительных ко времени отклика .

Эволюция планировщика Linux отражает рост требований к масштабируемости: ранний планировщик со сложностью O(n) просматривал весь спи-сок готовых процессов, сменивший его O(1)-планировщик использовал масси-вы очередей по приоритетам и эвристики интерактивности, а начиная с ядра 2.6.23 применяется модель справедливого распределения – сначала CFS, затем EEVDF .

3.1.3 Механизм nice и веса процессов

Для обычных процессов относительная важность задаётся значением nice в диапазоне от −20 (наивысший приоритет) до +19 (низший приоритет). Чем «вежливее» процесс (больше nice), тем меньшую долю процессора он по-лучает при конкуренции .

Справедливый планировщик преобразует nice в вес процесса: соседние значения nice отличаются по весу примерно в 1,25 раза, так что разница в 5 единиц nice даёт отношение долей процессорного времени около 1:3, а в 10 единиц – около 1:10 . Доля процессорного времени процесса при конку-ренции пропорциональна его весу:

- nice 0 соответствует весу 1024;
- nice 5 – весу 335;
- nice 10 – весу 110;
- nice 19 – весу 15.

Понизить приоритет своего процесса (увеличить nice) может любой пользователь; повышение приоритета (уменьшение nice) требует привилегий суперпользователя или соответствующего лимита RLIMIT_NICE .

3.1.4 Средства наблюдения и управления планированием

Для работы с подсистемой планирования из пользовательского окружения предназначен набор стандартных утилит:

- ps и top – просмотр приоритетов работающих процессов (столбцы NI и PRI/PR);
- nice – запуск программы с заданным значением nice;
- renice – изменение значения nice уже работающего процесса, группы процессов или всех процессов пользователя;
- chrt – просмотр и назначение политики планирования и статического приоритета, а также вывод поддерживаемых диапазонов приоритетов (ключ -m);
- taskset – просмотр и установка маски процессорной привязки (CPU affinity), ограничивающей множество ядер, на которых может выполняться процесс.

На уровне программного интерфейса перечисленным утилитам соответствуют системные вызовы getpriority и setpriority (значение nice), sched_setscheduler и sched_setattr (политика и параметры планирования),

sched_setaffinity (процессорная привязка) . Таким образом, всё, что демонстрируется в настоящей работе утилитами командной строки, доступно и программно из любого языка с доступом к системным вызовам.

3.2 Были исследованы приоритетов процессов.

Работа выполнялась на рабочей станции под управлением ОС Fedora Linux. Использовались стандартные средства дистрибутива: утилита ps из пакета procps-ng, утилиты renice, taskset и chrt из пакета util-linux, утилита nice из пакета coreutils, а также интерпретатор Python 3. Все действия выполнялись от имени непривилегированного пользователя.

Для просмотра приоритетов процессов использована утилита ps с поля-ми PID, NI (значение nice), PRI (приоритет в представлении ps) и именем команды, с сортировкой по приоритету. Результат выполнения команды приве-дён на рисунке 1.

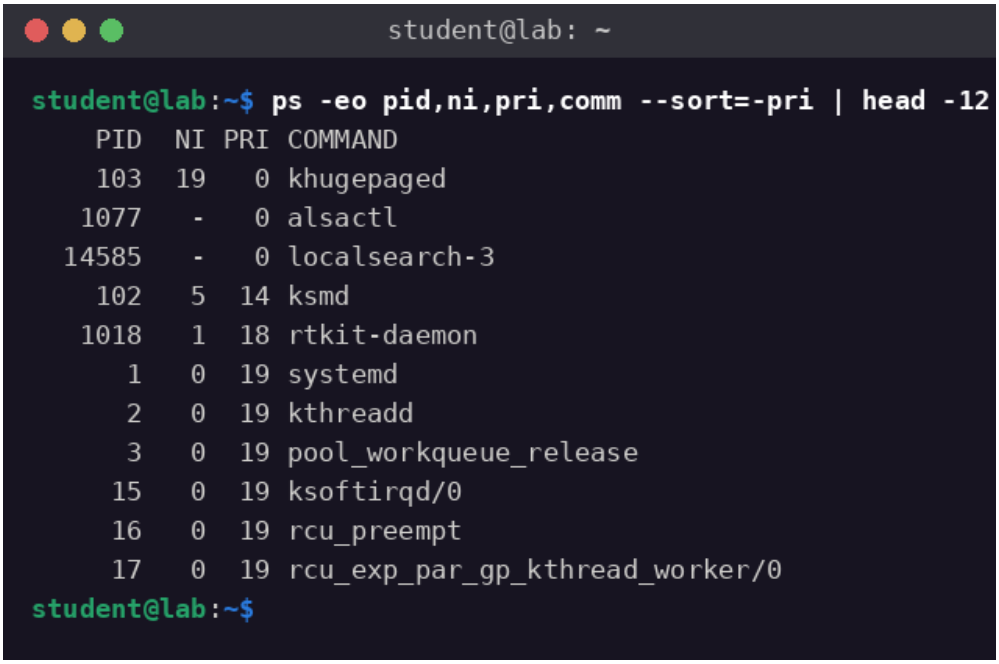


Рисунок 1 — Приоритеты процессов в системе (вывод ps)

Из вывода видно соотношение между значением nice и отображаемым приоритетом: для обычных процессов PRI = 19 − NI. Так, фоновый поток ядра khugepaged с nice 19 имеет PRI 0, поток ksmd с nice 5 – PRI 14, демон rtkit-daemon с nice 1 – PRI 18, а процессы с nice 0 (systemd, kthreadd и другие) – PRI

19. У процессов alsactl и localsearch-3 в столбце NI стоит прочерк: они выполняются с политикой, для которой значение nice не применяется.

Обращает на себя внимание и состав низкоприоритетных процессов: в верхних строках вывода (наименьший PRI) оказались служебные потоки ядра khugepaged и ksmd, выполняющие фоновую оптимизацию памяти (сборку огромных страниц и дедупликацию страниц соответственно). Такие задачи сознательно запускаются с повышенным nice, чтобы не отнимать процессор у пользовательских программ. Это наглядно показывает, как назначение приоритетов используется самим ядром для разделения работы на срочную и фоновую.

3.3 Было выполнено управление приоритетом с помощью утилит: nice и renice

Утилита nice позволяет запустить процесс с заданным значением nice, а renice – изменить его у уже работающего процесса. Для демонстрации фоновый процесс sleep был запущен со значением nice 10, после чего его приоритет был понижен до 19. Ход демонстрации приведён на рисунке 2.

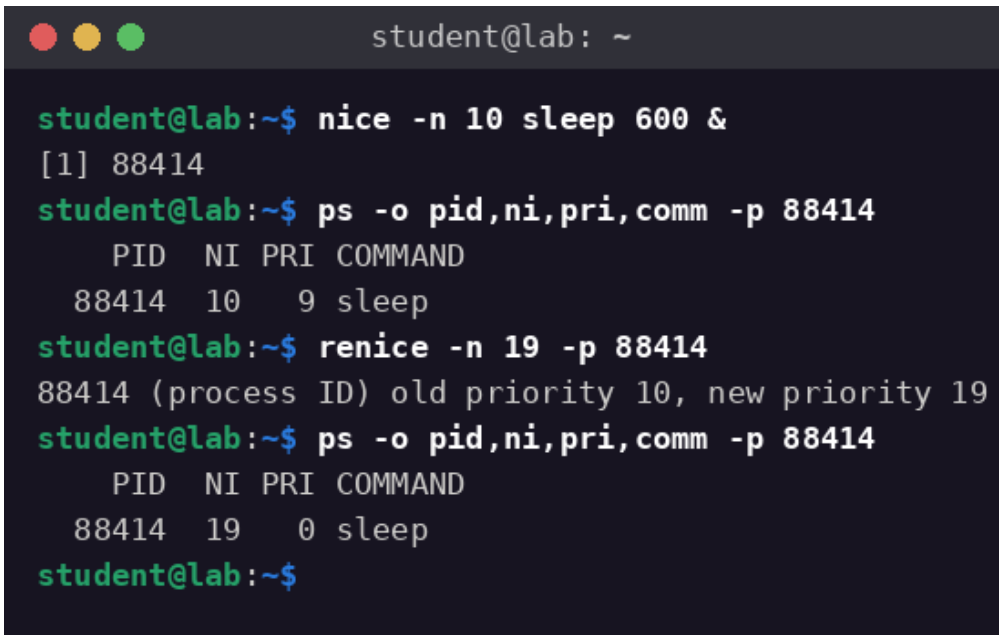


Рисунок 2 — Запуск процесса с nice 10 и изменение приоритета через renice

Процесс sleep получил PID 88414 и был запущен с NI 10, чему соответствует PRI 9. После выполнения команды renice -n 19 -p 88414 утилита

сообщила о смене приоритета (old priority 10, new priority 19), и повторный вызов ps подтвердил изменение: NI 19, PRI 0. Понижение приоритета выполнено от имени обычного пользователя без дополнительных привилегий, что согласуется с правилами, описанными в разделе теоретических сведений.

На практике описанный механизм применяется для длительных фоновых задач – компиляции больших проектов, архивирования, резервного копирования, пакетной обработки данных: запуск вида nice -n 19 команда позволяет выполнять тяжёлую работу, практически не влияя на отзывчивость интерактивных приложений. Обратная операция – повышение приоритета через отрицательные значения nice – доступна только суперпользователю и применяется, например, для процессов, обслуживающих воспроизведение звука или интерактивный ввод .

3.4 Были определены диапазоны приоритетов, поддерживаемые политиками плани-рования, с помощью утилиты chrt.

Диапазоны статических приоритетов, поддерживаемые политиками планирования данной системы, получены командой chrt -m. Результат приведён на рисунке 3.

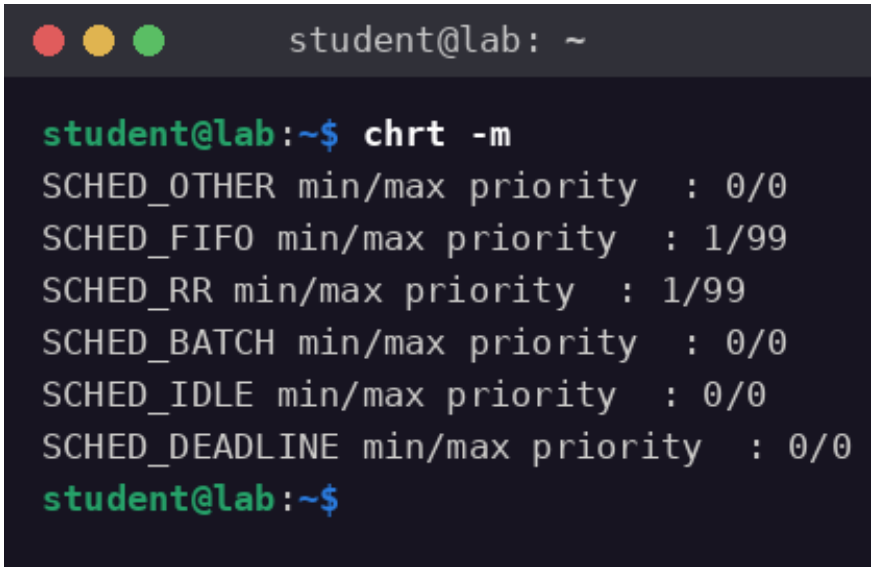


Рисунок 3 — Диапазоны приоритетов политик планирования (вывод chrt -m)

Полученные диапазоны подтверждают теоретические сведения: политики реального времени SCHED_FIFO и SCHED_RR используют статические

приоритеты от 1 до 99, тогда как у всех остальных политик статический приоритет равен 0 – конкуренция внутри них разрешается другими механизмами (весами nice для справедливого класса, параметрами runtime/deadline/period для SCHED_DEADLINE). Сводная характеристика политик приведена в таблице 1.

Таблица 1 – Политики планирования ОС Linux

Политика	**Класс**	**Приоритеты**	**Назначение**
SCHED_OTHER	справедливый	0	обычные процессы
SCHED_BATCH	справедливый	0	пакетные (неинтерактивные) задачи
SCHED_IDLE	справедливый	0	фоновые задачи низшего приоритета
SCHED_FIFO	реального времени	1–99	задачи РВ без квантования
SCHED_RR	реального времени	1–99	задачи РВ с круговым квантованием
SCHED_DEADLINE	deadline	0	задачи с гарантией срока выполнения

3.5 Была определена экспериментальная оценка влияния nice

Для количественной оценки влияния nice на распределение процессорного времени написана программа на языке Python, выполняющая счётный цикл в течение фиксированного интервала времени и выводящая число совершённых итераций. Число итераций пропорционально полученному процессом процессорному времени. Текст программы приведён ниже.

```
import sys, time

start = float(sys.argv)
while time.time() < start:
    pass

deadline = time.time() + 8.0
count = 0
while time.time() < deadline:
```

```
    count += 1
print(count)
```

Аргументом программе передаётся момент синхронного старта: оба конкурирующих экземпляра дожидаются одного и того же момента времени и лишь затем начинают счёт, что исключает влияние неодновременного запуска. В каждом опыте два экземпляра программы привязывались к одному ядру процессора утилитой taskset и запускались с разными значениями nice: у первого процесса nice 0, у второго – 5, 10 или 19. Команды запуска одного опыта приведены в листинге 2.

Привязка обоих процессов к одному ядру принципиальна для чистоты эксперимента: на многоядерной системе планировщик просто развёл бы конкурентов по разным ядрам, и каждый получил бы процессор целиком независимо от приоритета. Кроме того, работа на одном ядре нейтрализует влияние динамического изменения тактовой частоты – изменение частоты одинаково сказывается на обоих процессах и не искажает отношение числа итераций.

```
st=$(python3 -c "import time; print(time.time()+1.5)")
taskset -c 0 nice -n 0  python3 cpu_burn.py "$st" > result_hi.txt
&
taskset -c 0 nice -n 10 python3 cpu_burn.py "$st" > result_lo.txt
&
wait
```

Результаты трёх опытов сведены в таблице 2. Для каждого опыта указано число итераций обоих процессов, измеренная доля процессорного времени низкоприоритетного процесса и теоретическая доля, рассчитанная по весам планировщика из раздела теоретических сведений.

Таблица 2 – Распределение процессорного времени при конкуренции процессов

nice второго процесса	Итераций (nice 0)	Итераций (второй)	Доля второго, %	Теоретическая доля, %
5	4 596 424	1 495 306	24,5	24,6
10	5 452 416	584 181	9,7	9,7
19	5 992 836	100 079	1,6	1,4

Измеренные доли практически совпали с теоретическими, рассчитанными из весов nice справедливого планировщика: при nice 5 низкоприоритетный процесс получил около четверти процессорного времени, при nice 10 – около десятой части, а при nice 19 – менее двух процентов. Небольшое расхождение в опыте с nice 19 объясняется фоновой активностью системы и накладными расходами интерпретатора. Эксперимент наглядно подтверждает, что значение nice управляет именно пропорцией распределения процессора, а не абсолютным запретом на выполнение: даже процесс с nice 19 продолжает получать процессорное время.

При интерпретации результатов следует учитывать механизм автогруппировки (autogrouping): при включённом параметре ядра sched_autogroup_enabled справедливое распределение процессора сначала выполняется между сеансами, и значения nice отдельных процессов действуют лишь внутри своей группы . В проведённом эксперименте оба конкурирующих процесса запускались из одного сеанса и принадлежали одной группе, поэтому веса nice применялись к ним непосредственно и результаты корректно отражают модель весов планировщика.

4 Вывод: В результате выполнения работы были изучены механизмы планирования процессов в операционной системе Linux, исследовано влияние значения nice на приоритет и фактическое распределение процессорного времени, проведено ознакомление с политиками планирования реального времени.